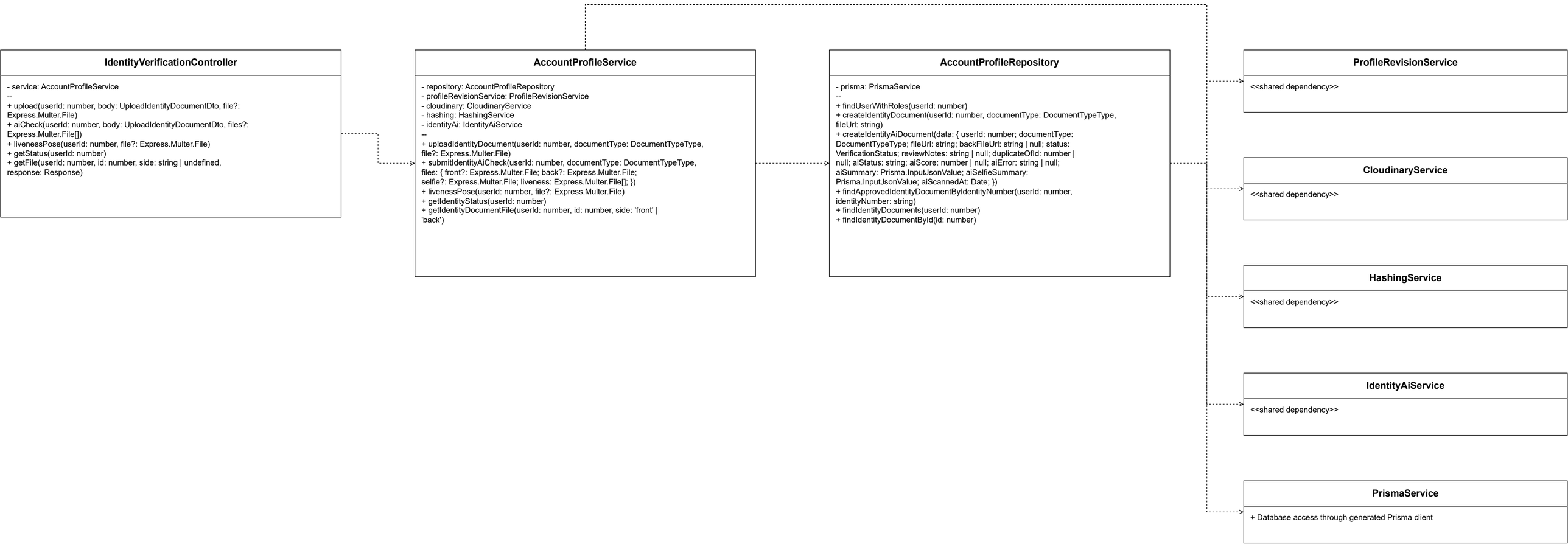
IdentityVerificationController
- service: AccountProfileService
--
+ upload(userId: number, body: UploadIdentityDocumentDto, file?: Express.Multer.File)
+ aiCheck(userId: number, body: UploadIdentityDocumentDto, files?: Express.Multer.File[])
+ livenessPose(userId: number, file?: Express.Multer.File)
+ getStatus(userId: number)
+ getFile(userId: number, id: number, side: string | undefined, response: Response)
AccountProfileService
- repository: AccountProfileRepository
- profileRevisionService: ProfileRevisionService
- cloudinary: CloudinaryService
- hashing: HashingService
- identityAi: IdentityAiService
--
+ uploadIdentityDocument(userId: number, documentType: DocumentTypeType, file?: Express.Multer.File)
+ submitIdentityAiCheck(userId: number, documentType: DocumentTypeType, files: { front?: Express.Multer.File; back?: Express.Multer.File; selfie?: Express.Multer.File; liveness: Express.Multer.File[]; })
+ livenessPose(userId: number, file?: Express.Multer.File)
+ getIdentityStatus(userId: number)
+ getIdentityDocumentFile(userId: number, id: number, side: 'front' | 'back')
AccountProfileRepository
- prisma: PrismaService
--
+ findUserWithRoles(userId: number)
+ createIdentityDocument(userId: number, documentType: DocumentTypeType, fileUrl: string)
+ createIdentityAiDocument(data: { userId: number; documentType: DocumentTypeType; fileUrl: string; backFileUrl: string | null; status: VerificationStatus; reviewNotes: string | null; duplicateOfId: number | null; aiStatus: string; aiScore: number | null; aiError: string | null; aiSummary: Prisma.InputJsonValue; aiSelfieSummary: Prisma.InputJsonValue; aiScannedAt: Date; })
+ findApprovedIdentityDocumentByIdentityNumber(userId: number, identityNumber: string)
+ findIdentityDocuments(userId: number)
+ findIdentityDocumentById(id: number)
ProfileRevisionService
<<shared dependency>>
CloudinaryService
<<shared dependency>>
HashingService
<<shared dependency>>
IdentityAiService
<<shared dependency>>
PrismaService
+ Database access through generated Prisma client